Design Report — Port Scanning Activity

Group: Charles, Axel, André

1. Code structure

The code follows a modular architecture. This approach is preferred to ensure readability and enhance maintainability.

```
project-root/
│
├── .env            # Input values
├── config.py       # Logic to load and validate .env
├── utils.py        # Useful tools
└── main.py         # Main script
```

The structure is composed of the following elements:

- Configuration layer ([config.py]())

  Centralizes all environment variables and initial validation. We use error handling to raise exceptions to improve the functionality. Below you can find the function used in config.py :

  `load_config() -> dict`:

  Purpose: To bridge the gap between the `.env` file (strings) and the Python environment (objects/integers)

  Logic: It pulls raw strings from the `.env` file and transforms them into usable Python objects.

- Logic Layer ([utils.py]())

  Contains the reusable networking tools. This separates the *mechanism* of scanning (sockets/ping) from the *workflow* of the application. In the logic layer we developed two functions **is_host_alive** and **scan_port**.

  `is_host_alive(ip: str)`:

Purpose: To perform "Host Discovery."

Logic: It executes a system-level Ping. If the host doesn't respond, it returns `False`, allowing the main program to skip that IP entirely.

`scan_port(ip: str, port: int, timeout: int)`:

Purpose: To perform the "TCP Handshake" attempt.

Logic: It attempts to open a stream socket to a specific IP and port. It translates the numerical result of the `connect_ex` call into a human-readable status (`OPEN`, `CLOSED`, or `FILTERED`).

- Main Layer (main.py)

  Acts as the engine. It manages the loop through the IP range, handles the CSV output, and coordinates between utils.py and config.py.

  `run_scanner()`:

  Purpose: To manage the scan.

  Flow Control: It handles the "Big Loop"—iterating through every IP and every port in the user's range.

  Reporting: It uses the `csv.writer` to save data to the disk and aggregates the final statistics (counts of open/closed ports) to display a summary to the user once the task is finished.

2. Libraries and dependencies

The project relies on a mix of standard Python libraries and one external package (dotenv) to handle specialized tasks:

Library	Purpose	Functionality in Project
socket	Network Interface	Used in `scan_port` to create TCP connections and check for open ports
ipaddress	IP Manipulation	Converts strings to IPv4 objects and calculates IP ranges using `summarize_address_range`
subprocess	OS Interaction	Executes the system `ping` command to verify if a host is "alive" before scanning
dotenv	Configuration	Loads variable data from a `.env` file, keeping code separate from environment settings
csv	Data Export	Ensures all scan results are logged in a structured format for later analysis
errno	Error Mapping	Maps numerical socket return codes to human-readable states (e.g., `ECONNREFUSED` = `CLOSED`)

3. Wireshark observations before building

What protocol is used for host discovery? What does the traffic look like?

Since the Metasploitable VM and Linux VM are on the same local network, Nmap uses ARP (Address Resolution Protocol) for host discovery.

ARP is used on local networks to determine the MAC address associated with an IP address. It is the fastest and most reliable way for Nmap to verify whether a host is online on the same LAN.

In Wireshark, traffic looks like: Who has 192.168.1.X? Tell 192.168.1.Y
Meaning: "Who owns this IP address?"

Metasploitable answers: 192.168.1.X is at 08:00:27:xx:xx:xx
Meaning:"I am that host."

126	2026-03-13 14:12:40.217658673	Parallels_8d:dc:17	Broadcast	ARP	42	Who has 192.168.1.22? Tell 192.168.1.17
127	2026-03-13 14:12:40.217678798	Parallels_8d:dc:17	Broadcast	ARP	42	Who has 192.168.1.23? Tell 192.168.1.17
128	2026-03-13 14:12:40.217680673	Parallels_8d:dc:17	Broadcast	ARP	42	Who has 192.168.1.24? Tell 192.168.1.17
129	2026-03-13 14:12:40.217682090	Parallels_8d:dc:17	Broadcast	ARP	42	Who has 192.168.1.25? Tell 192.168.1.17
130	2026-03-13 14:12:40.217683465	Parallels_8d:dc:17	Broadcast	ARP	42	Who has 192.168.1.26? Tell 192.168.1.17
131	2026-03-13 14:12:40.217684923	Parallels_8d:dc:17	Broadcast	ARP	42	Who has 192.168.1.27? Tell 192.168.1.17
132	2026-03-13 14:12:40.217686465	Parallels_8d:dc:17	Broadcast	ARP	42	Who has 192.168.1.28? Tell 192.168.1.17
133	2026-03-13 14:12:40.217691423	Parallels_8d:dc:17	Broadcast	ARP	42	Who has 192.168.1.29? Tell 192.168.1.17
134	2026-03-13 14:12:40.218678548	ee:23:c7:60:ae:0f	Parallels_8d:dc:17	ARP	42	192.168.1.28 is at ee:23:c7:60:ae:0f
135	2026-03-13 14:12:40.221334007	Parallels_8d:dc:17	Broadcast	ARP	42	Who has 192.168.1.32? Tell 192.168.1.17
136	2026-03-13 14:12:40.221348007	Parallels_8d:dc:17	Broadcast	ARP	42	Who has 192.168.1.33? Tell 192.168.1.17

What does a connection attempt to an open port look like?

Since the Metasploitable VM and the Linux VM are on the same local network, Nmap sends a SYN packet to the target port to initiate a connection.
If the port is open, the Metasploitable VM machine responds with a SYN/ACK packet.
Since Nmap is only scanning and not establishing a full connection, it immediately resets the connection(send RST packet).

(Check screenshoot below, for port 22)

No.	Time	Source	Destination	Protocol	Length	Info
79	2026-03-14 17:49…	192.168.1.17	192.168.1.28	TCP	58	35084 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
94	2026-03-14 17:49…	192.168.1.28	192.168.1.17	TCP	58	22 → 35084 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460
95	2026-03-14 17:49…	192.168.1.17	192.168.1.28	TCP	54	35084 → 22 [RST] Seq=1 Win=0 Len=0

Can you spot the scanning pattern? What makes it recognizable as a scan?

Yes, the scanning pattern can be clearly identified in Wireshark.
When running nmap, Nmap sends many TCP SYN packets in rapid succession to different ports on the same host.

No.	Time	Source	Destination	Proto	Length	Info
100	2026-03-14 17:49:57.869630851	192.168.1.17	192.168.1.28	TCP	58	35084 → 97 [SYN] Seq=0 Win=1024
101	2026-03-14 17:49:57.869721726	192.168.1.17	192.168.1.28	TCP	58	35084 → 13 [SYN] Seq=0 Win=1024
102	2026-03-14 17:49:57.869817142	192.168.1.17	192.168.1.28	TCP	58	35084 → 100 [SYN] Seq=0 Win=1024
103	2026-03-14 17:49:57.869922309	192.168.1.17	192.168.1.28	TCP	58	35084 → 59 [SYN] Seq=0 Win=1024
104	2026-03-14 17:49:57.869960059	192.168.1.17	192.168.1.28	TCP	58	35084 → 62 [SYN] Seq=0 Win=1024
105	2026-03-14 17:49:57.870087267	192.168.1.17	192.168.1.28	TCP	58	35084 → 72 [SYN] Seq=0 Win=1024
106	2026-03-14 17:49:57.870192726	192.168.1.17	192.168.1.28	TCP	58	35084 → 47 [SYN] Seq=0 Win=1024
107	2026-03-14 17:49:57.870336559	192.168.1.17	192.168.1.28	TCP	58	35084 → 99 [SYN] Seq=0 Win=1024
108	2026-03-14 17:49:57.870502976	192.168.1.17	192.168.1.28	TCP	58	35084 → 90 [SYN] Seq=0 Win=1024
109	2026-03-14 17:49:57.870604434	192.168.1.17	192.168.1.28	TCP	58	35084 → 33 [SYN] Seq=0 Win=1024
110	2026-03-14 17:49:57.870856601	192.168.1.17	192.168.1.28	TCP	58	35084 → 3 [SYN] Seq=0 Win=1024 L
111	2026-03-14 17:49:57.870962601	192.168.1.17	192.168.1.28	TCP	58	35084 → 31 [SYN] Seq=0 Win=1024
112	2026-03-14 17:49:57.871068851	192.168.1.17	192.168.1.28	TCP	58	35084 → 26 [SYN] Seq=0 Win=1024
113	2026-03-14 17:49:57.871256892	192.168.1.17	192.168.1.28	TCP	58	35084 → 70 [SYN] Seq=0 Win=1024
114	2026-03-14 17:49:57.871334309	192.168.1.17	192.168.1.28	TCP	58	35084 → 91 [SYN] Seq=0 Win=1024
115	2026-03-14 17:49:57.871450767	192.168.1.17	192.168.1.28	TCP	58	35084 → 1 [SYN] Seq=0 Win=1024 L
116	2026-03-14 17:49:57.871536476	192.168.1.17	192.168.1.28	TCP	58	35084 → 75 [SYN] Seq=0 Win=1024
117	2026-03-14 17:49:57.871585517	192.168.1.17	192.168.1.28	TCP	58	35084 → 46 [SYN] Seq=0 Win=1024
118	2026-03-14 17:49:57.871645226	192.168.1.17	192.168.1.28	TCP	58	35084 → 64 [SYN] Seq=0 Win=1024

4. System Refinements & Future Considerations

This section details the design choices made during the development phase to ensure the scanner is both robust and legally compliant within a testing environment.

4.1 Reliability and Error Handling

A portion of the development was dedicated to exception handling. By categorizing potential points of failure, the tool provides clear feedback rather than generic crashes:

- Configuration Guardrails: The config.py module acts as a validator, ensuring that port ranges are logically sound (Start < End) and valid the type of the input.

4.2. Performance Optimization

The "Ping-before-Scan" logic improved the functionality of the code:

In network environments where many IP addresses may be unassigned, attempting to scan 65535 ports on a dead host is a massive waste of resources. By verifying a host's "Alive" status via ICMP Echo.

4.3. Future Scope

While the current version is functional, the design allows for future enhancements:

- Multithreading: The modular structure of scan_port makes it ready to be integrated into a threaded loop, which would allow for scanning multiple ports simultaneously.
- Service Versioning: Future iterations could include a "Banner Grabbing" module in utils.py to identify not just if a port is open, but what service (and version) is running behind it.

5. Conclusion

The resulting tool is a modular, typed, and well-documented network scanner. By bridging the gap between low-level socket programming and high-level Python abstractions, the project successfully demonstrates how security tools can be built to be both efficient and maintainable.